Programación de aplicaciones de red



GP22_1

Programación con el interfaz sockets

En esta práctica se va a trabajar sobre la competencia de programación de aplicaciones, en concreto programando una aplicación servidora empleando la interfaz que nos proporcionan los sockets.

Programación con el interfaz Socket

En esta práctica se revisará el funcionamiento de un servicio de ECO, en donde el servidor devolverá la misma cadena que le ha enviado un cliente. La tarea propuesta para la práctica consiste en terminar el código existente (puede disponer de una copia en Aula Virtual) correspondiente al servidor conectivo. No hará falta implementar el cliente dado que nos va a valer con un cliente `telnet`.

El alumno deberá incorporar las sentencias que sean necesarias sustituyendo las "***XXXXXXX***" que aparecen en el código para que pueda haber una comunicación entre el cliente (empleando el cliente telnet) y el servidor realizado. Como ayuda se remite a la ayuda oficial de Python[1] y el diagrama de sockets TCP que puede encontrar al final de este documento.

Las comunicaciones se realizarán mediante el protocolo TCP, sobre un puerto a determinar por el alumno. Para evitar tener problemas en la escucha del servidor, el puerto tendrá siempre un valor superior a 5000. Para la conexión desde el cliente, se utilizarán sockets, que en Python se manejan a través de la librería `socket`[2].

A continuación, se tiene la parte del código del servidor de ECO.

```
import socket
import sys

HOST = ''  # Nombre o direccion de la maquina. Puede estar vacio en el servidor
PORT = 8888  # Puerto de escucha del servidor. PORT >= 5000
BUFFER = 1024 # Tamano del buffer de recepcion

# Primero se crea el socket
s = socket.XXXXXXX(socket.AF_INET, socket.SOCK_STREAM)
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

print('Socket creado...')

# Enlaza el socket con la direccion y puerto local
try:
	s.bind((XXXXXXX, XXXXXXX))

# Captura excepciones. Si existe, un error sale del programa
except socket.error as msg:
	print('Fallo en BIND. Cod. Error: {} Mensaje: {}'.format(msg.errno,msg.strerror))
	sys.exit()

print('Socket Enlazado...')

# Comienza la escucha en el socket
s.listen(10)
print('Socket escuchando...')

while 1:
	try:
		# Espera a que lleguen clientes. Se aceptan conexiones.
		cliente, addr = s.XXXXXXX()
		print('Conectado con {}:{}'.format(addr[0],addr[1]))

		while 1:
```

[1] https://docs.python.org/3/
[2] https://docs.python.org/3/library/socket.html

```
                        # Recepcion de datos. Se reciben datos del cliente.
                        datos = cliente.XXXXXXX(XXXXXXX)
                        if datos:
                    # Se envia al cliente lo mismo que se ha recibido (eco).
                                cliente.XXXXXXX(XXXXXXX)
                                # Se imprime pasando un byte string a string
                                print('Recibido: {}'.format(datos.decode("utf-8")))
                         # Se dejara de trabajar con el cliente si se recibe EXIT
                                # Tambien lo podemos comparar si lo hacemos con un byte string
                                #(los dos tipos deben ser iguales)
                                if datos[0:4] == b'EXIT':
                                        cliente.close()
                                        break
        except KeyboardInterrupt:
                print('\nCerrando socket y apagando servidor...')
                s.close()
                break

s.close()
```

El código anterior se puede encontrar en el archivo `servidor_a_completar_Python3.py`. Se pide al alumno modificar el código cambiando las cadenas `xxxxxxx` por código correcto que permita conectar con el servidor.

A la hora de incorporar el código hay que tener en cuenta especialmente:

- Las llaves en Python se implementan a través de la **indentación (introduciendo tabuladores).**
- **No se deben poner acentos en el código**, aunque sea en los comentarios.

Para ejecutar el código, se debe utilizar el intérprete de Python **en un terminal**:

```
$ python <nombre_del_programa.py>
```

Por ejemplo, para el código del servidor, debidamente modificado (desde la carpeta donde está el archivo):

```
$ python servidor_a_completar_Python3.py
```

Para conectar un cliente telnet con el servidor, se debe utilizar **en otro terminal** la orden telnet, seguida de la dirección (127.0.0.1 si estamos ejecutando cliente y servidor en la misma máquina) y del puerto del servidor (el configurado en el código del servidor):

```
$ telnet 127.0.0.1 8888
```

Para desconectar el *cliente* del servidor se debe enviar la palabra "EXIT" **(en mayúsculas)**. El *servidor* se puede detener utilizando la combinación de teclas **Ctrl+C.**

Tras la realización del código y verificar que funciona (esto es, es posible conectar con el servidor a través de `telnet` y se pueden enviar y recibir mensajes), se pide contestar a las siguientes preguntas:

1. Al arrancar el servidor, éste imprime una línea que comienza por “Conectado con” ¿Qué son los datos que aparecen a continuación?

2. Conectar dos clientes al mismo servidor (tras parar y rearrancar el servidor, en dos terminales diferentes, ejecutar `telnet` dos veces). Cerrar la conexión del primer cliente arrancado. ¿Cuál es el comportamiento del servidor? ¿A qué se debe? ¿Sería posible conectar más de un cliente a través de un socket?

3. Después de rearrancar el servidor de nuevo, conectarse al mismo desde un navegador web (usando la dirección y el puerto del servidor, separados por “:” como dirección, *p.ej. http://127.0.0.1:8888*). ¿Qué ocurre en el servidor? ¿Qué se ve en el navegador?

Al finalizar este trabajo, cargar un archivo comprimido a la actividad correspondiente del Aula Virtual, conteniendo:

- Código fuente del servidor completado.
- Documento con las preguntas contestadas.

DIAGRAMA SOCKETS TCP

